

Human-in-the-Loop & Workflows

LangGraph Session Notes

Topics: Human-in-Loop from Scratch · interrupt() · Command · Approval Flow · Coming Next

❖ 1. What is Human-in-the-Loop?

Human-in-the-loop = **pausing an agent workflow to get human approval before proceeding.**

The agent proposes an action → workflow pauses → human reviews → approves or rejects → workflow resumes or stops.

Real example — Copilot:

When Copilot wants to make changes to your code, it always asks:

Do you want me to make this change? Yes or No

That's human-in-the-loop. The agent does not act without approval.

When to use it

- **High-risk actions** — sending emails, making payments, deleting data
- **Irreversible actions** — deploying code, calling external APIs with side effects
- **Uncertain decisions** — agent is not confident and needs human judgment
- **Compliance requirements** — regulated industries where human sign-off is mandatory

Rule: If the action cannot be undone easily — add a human checkpoint.

❖ 2. From Scratch vs LangGraph Library

From Scratch

Manual implementation using plain Python.

You write the pause, the input(), and the resume logic yourself.

Shows exactly what happens under the hood.

```
action = propose_action()

approval = input('Approve? y/n')

if approval == 'y':

    execute(action)

else:

    print('Action rejected')
```

LangGraph Library

Uses built-in interrupt() and Command.

State is saved automatically via checkpointer.

Graph can resume from exact pause point.

```
from langgraph.types import

interrupt, Command

def review_node(state):

    human_input = interrupt(

        state['proposed_action']

    )

    return {'approval': human_input}
```

❖ 3. interrupt() and Command — How They Work

interrupt(value)

Pauses graph execution at that exact point.

Sends value to the human as a question or prompt.

Saves the entire graph state via checkpointer.

Waits until human provides input.

Command(resume=value)

Resumes the graph from the paused point.

Passes the human's response back into the graph.

The `interrupt()` call returns the human's input.

```
# First invoke – graph pauses at interrupt
```

```
graph.invoke(state, config)
```

```
# Human provides input – graph resumes
```

```
graph.invoke(Command(resume='approved'), config)
```

★ **Critical requirement: Checkpointer is MANDATORY for `interrupt()` to work.**

Without checkpointer → state is lost when graph pauses → cannot resume.

With checkpointer → state saved → graph resumes from exact pause point.

```
graph = builder.compile(checkpointer=MemorySaver())
```

❖ 4. The Approval Workflow — Step by Step

User Query

↓

Agent Node → generates proposed action

↓

Review Node → `interrupt()` – sends action to human

↓ ← PAUSED – waiting for human

Human reviews proposed action

↓

Human approves → `Command(resume='approved')`

↓

Execute Node → executes the action

↓

END — action completed

Human rejects → `Command(resume='rejected')`

↓

END — action cancelled

Your mentor's example: Refund of \$5,000 for customer — agent proposes, human approves, then executes.

❖ 5. Key Imports

```
from typing import TypedDict, Literal
```

```
from pydantic import BaseModel
```

```
from langchain_openai import ChatOpenAI
```

```
from langgraph.graph import StateGraph, START, END
```

```
from langgraph.checkpoint.memory import MemorySaver
```

```
from langgraph.types import interrupt, Command
```

The two new imports compared to previous sessions:

→ `interrupt` — pauses the graph

→ `Command` — resumes the graph with human input

❖ 6. Coming in Next Sessions

Your mentor said next 2 sessions are dedicated to:

- **Subgraphs** — full graph used as a node inside another graph
- **Streaming API** — stream agent output in real time
- **Multi-agent workflows** — multiple agents working together
- **MCP (Model Context Protocol)** — standardized tool calling
- **Evaluation & guardrails** — testing agent quality
- **Agentic RAG project** — end-to-end project combining everything

After all LangGraph concepts → full project + deployment.

Your mentor said: 99% of LangGraph will be done after subgraphs and multi-agent.

❖ 7. Quick Reference

interrupt(value) = pause graph, send value to human, wait for response

Command(resume=v) = resume graph from pause point with human's response v

MemorySaver = required checkpointer — saves state when graph pauses

Approval flow: Agent proposes → interrupt() → human reviews → Command(resume) → execute or reject

From scratch = manual input() + if/else — shows mechanics

LangGraph = interrupt() + Command + checkpointer — production ready

★ Rule: No checkpointer = no resume. Always compile graph with MemorySaver when using interrupt().